

2do Parcial: Puntos en el plano

Datos como texto

Es común recibir datos numéricos como cadenas de texto. Para trabajar con ellos hay que *parsearlos*: convertirlos a un tipo de datos adecuado.

Por ejemplo, dada la cadena de caracteres `cadena`:

```
cadena = '1, 2\n3, 4'
filas = cadena.split('\n')      # ['1, 2', '3, 4'] (separa filas)
xy = filas[0].split(',')        # ['1', '2']      (obtiene cada número por separado)
tupla = (float(xy[0]), float(xy[1])) # (1.0, 2.0)
```

1: (2 puntos)

Parsear

Escribí una función `parsear(cadena)` que reciba la cadena de arriba y devuelva una lista de tuplas de floats.

Resultado esperado:

```
[ (0.8, 0.2), (0.1, 0.5), (0.6, 0.6), (0.2, 0.1), (0.4, 0.9), (0.3, 0.3) ]
```

```
cadena = """0.8, 0.2 0.1, 0.5 0.6, 0.6 0.2, 0.1 0.4, 0.9 0.3, 0.3"""
```

```
def parsear(cadena): # Tu código !
```

```
puntos = parsear(cadena) print(puntos)
```

2 (3 puntos)

Distancia al origen

La distancia euclidiana de un punto (x, y) al origen es:

$$d = \sqrt{x^2 + y^2}$$

Escribí una función `distancias(puntos)` que reciba la lista de tuplas y devuelva un diccionario donde cada clave es una tupla y su valor es la distancia al origen.

Resultado esperado:

```
{(0.8, 0.2): 0.825..., (0.1, 0.5): 0.509..., ...}
```

La raíz cuadrada se puede calcular como `x ** 0.5`. `def distancias(puntos): # Tu código acá`

```
dists = distancias(puntos) for punto, d in dists.items(): print(punto, '->', round(d, 3)) # (round redondea a 3 decimales para claridad)
```

3 (4 puntos)

Centroide

El centroide de un conjunto de puntos es el punto promedio: se calcula como el promedio de todas las coordenadas `x` y el promedio de todas las coordenadas `y`.

Escribí una función `centroide(puntos)` que devuelva una tupla `(cx, cy)` con las coordenadas del centroide.

Resultado esperado para la cadena de arriba:

```
(0.4, 0.433)
```

```
def centroide(puntos): # Tu código acá
```

```
cx, cy = centroide(puntos) print(f'Centroide: ({round(cx, 3)}, {round(cy, 3)})')
```

4 (6 puntos)

Orden por distancia

Escribí una función `orden_distancia(puntos)` que devuelva una lista de enteros indicando el orden de los puntos de menor a mayor distancia al origen.

Por ejemplo, si el punto más cercano al origen es el de la posición `3`, el primer elemento de la lista será `3`.

Resultado esperado para la cadena de arriba:

```
[3, 5, 1, 0, 2, 4]
```

Pista: podés usar el diccionario `dists` del ejercicio anterior. `def orden_distancia(puntos): # Tu código acá`

```
orden = orden_distancia(puntos) print(orden) print('Del más cercano al más lejano:') for i in orden: print(f' {i}: {puntos[i]} -> {round(dists[puntos[i]], 3)}')
```